

Problem A. Welcome to NPCAPC

Let N be the length of the string.

For $N \leq 2 \cdot 10^5$, use the DP

$$\text{DP}[i][j][k],$$

where i is the number of decided characters, j is how many characters of `NPCAPC` have already been matched, and k is how many characters of `npcapc` have already been matched. The transition chooses the next character and updates the two matched-prefix lengths. The number of states is $(N + 1) \cdot 7 \cdot 7$.

For $N \leq 10^9$, compress the DP into a 49×49 transition matrix. A state is (a, b) with $0 \leq a, b < 7$, encoded as $7a + b$. The transition matrix G is obtained from the one-character DP transition. Then the answer for a single query is the entry corresponding to paths of length N from state $(0, 0)$ to state $(6, 6)$, namely an entry of G^N , computable in $\mathcal{O}(49^3 \log N)$ time.

For the full solution with many queries, direct matrix exponentiation per query is too slow. By Cayley–Hamilton, for any fixed entry of G^t , the sequence in t satisfies a linear recurrence of degree at most 49. Compute the first $2 \cdot 49$ terms, recover the rational generating function

$$F(x) = \frac{A(x)}{B(x)}, \quad \deg A < 49, \quad \deg B \leq 49,$$

by solving the linear equations from $F(x)B(x) = A(x)$ modulo x^{98} , and then answer each query by Bostan–Mori. This gives about $\mathcal{O}(M \log M \log N)$ per query for $M = 49$.

Problem B. Some Sum of Subset

Sort the sequence in non-increasing order, so assume

$$A_1 \geq A_2 \geq \dots \geq A_N.$$

For a subset $S = \{x_1 < x_2 < \dots < x_{|S|}\}$, define

$$f(S) = \{x_1, x_2, \dots, x_{|S|-k}\}.$$

Because of the order of A , this is one of the choices that maximizes the sum after removing k elements from S . Thus the condition on S can be rephrased as

$$\sum_{i \in f(S)} A_i \geq M.$$

Count by the value $T = f(S)$. If $T = \{x_1, \dots, x_{|T|}\}$ and $x_{|T|} = i$, then the elements removed from S must be chosen from $\{i + 1, i + 2, \dots, N\}$, so there are $\binom{N-i}{k}$ possible sets S that map to this T .

Let B_i be the number of valid T whose maximum index is i . Then the answer is

$$\sum_{i=1}^N B_i \binom{N-i}{k}.$$

The values B_i are obtained by subset-sum DP. Let $\text{dp}[i][j]$ be the number of subsets of $\{1, 2, \dots, i\}$ with sum less than j . Then

$$B_i = 2^{i-1} - \text{dp}[i-1][M - A_i].$$

With Pascal’s triangle for the binomial coefficients, the total complexity is $\Theta(NM + N^2)$.

Problem C. Solve with Friends

Fix that Namuka solves k problems and Napuka solves the remaining $N - k$. Then an exchange argument shows that Namuka should solve the k problems with the smallest values of $A_i - B_i$, and Napuka should solve the others.

Indeed, suppose Namuka solves problem i and Napuka solves problem j , but $A_i - B_i > A_j - B_j$. Swapping these two assignments changes the sum of solving times by

$$(A_i + B_j) - (A_j + B_i) = (A_i - B_i) - (A_j - B_j) > 0,$$

so the original assignment was not optimal.

Therefore sort all pairs (A_i, B_i) by $A_i - B_i$. For fixed k , the minimum total time is

$$X_k = \sum_{i=1}^k A_i + \sum_{i=k+1}^N B_i + \sum_{i=0}^{k-1} C_i + \sum_{i=0}^{N-k-1} D_i.$$

The answer is $\min_{0 \leq k \leq N} X_k$. Precompute prefix sums of A, B, C, D after sorting; then every X_k is computed in $\Theta(1)$ time. The total complexity is $\Theta(N \log N)$.

Problem D. Two Box

For the first subtask, use a bitmask DP:

$\text{dp}[i][j]$ = the number of ways after the first i operations, with black-box state j .

This costs $\mathcal{O}(N2^M MQ)$.

For $Q = 1$, decide the behavior of balls in the order $M, M-1, \dots, 1$. For ball k , look at each maximal interval $[l, r]$ in which ball k can be in the black box. In almost all cases, ball k must be selected an even number of times inside this interval; the exception is when the interval reaches the right end of all operations, where an odd number is also allowed.

For each maximal interval, store a generating function whose coefficient represents the number of remaining operations. After the choices for balls $k+1, k+2, \dots, M$ inside the interval are fixed, apply the even-use transition. In polynomial terms, choosing an arbitrary number of positions transforms $f(x)$ to $f(x+1)$, while choosing an even number transforms it to

$$\frac{f(x+1) + f(x-1)}{2}.$$

A direct convolution implementation gives an $\mathcal{O}(QNM \log N \log(N/M))$ -type solution.

For the full solution, observe that the final answer is $f(1)$ for the bottom interval. At depth k , it is enough to keep only the values

$$f(2-k), f(3-k), \dots, f(k),$$

so one interval needs only $\mathcal{O}(M)$ information. During an update, the maximal intervals can be handled as $\mathcal{O}(M)$ interval deletions and insertions. Prepare segment trees for every pair (k, v) with $1 \leq k \leq M$ and $2-k \leq v \leq k$; the monoid operation is multiplication of the stored values.

When adding an interval $[l, r]$ on level k , query all needed values from the level $k+1$ segment trees on $[l, r]$, apply the transformation above, and insert the resulting level- k values. Deletion just removes the corresponding values from the level- k segment trees. The total complexity is

$$\mathcal{O}((N+Q)M^2 \log N).$$

Problem E. Aim High

The editorial gives an explicit construction showing that height 4 is attainable. It starts from rows with one piece on every point, repeatedly moves pieces from lower rows to create fourteen consecutive points on row $y = 0$ with two pieces, and then uses these double pieces to move a piece up to row $y = 4$. The construction is symmetric on the left and right sides and finishes by combining the middle four columns.

It remains to prove that height at least 5 is impossible. Let the target point be $p = (5, k)$. For a piece at Manhattan distance d from p , define its potential as x^d , where

$$x = \frac{\sqrt{5}-1}{2}.$$

Consider one operation that consumes pieces at distances d and $d + 1$. The produced piece may be at distance $d - 1, d, d + 1$, or $d + 2$. The potential changes by one of

$$x^{d-1} - x^d - x^{d+1}, \quad -x^{d+1}, \quad -x^d, \quad x^{d+2} - x^{d+1} - x^d.$$

For the chosen value of x , none of these changes is positive, so total potential never increases.

If we even give ourselves one initial piece at every point with non-positive y -coordinate, the total potential toward p is

$$x^5 \sum_{i \geq 0} x^i \left(1 + \sum_{j \geq 1} 2x^j \right) = x^5(x + 2)(2x + 3) = 1.$$

The real initial state has strictly smaller potential, but placing a piece at p would require potential at least 1. Hence no point with $y \geq 5$ can be reached.

Problem F. Train Seats

Sort the positions first:

$$A_1 \leq A_2 \leq \dots \leq A_N.$$

For $N \leq 500$, an interval DP works: let $\text{dp}[i][j]$ be the maximum score obtained when seating people i through j . It has $\mathcal{O}(N^2)$ states and $\mathcal{O}(N)$ transitions per state, for $\mathcal{O}(N^3)$ time.

For $N \leq 3000$, the first seated person in an optimal solution for interval $[i, j]$ is always either i or j . If the first person were an interior k , compare the solution with the orders that seat the first chosen people of the left and right subintervals around k . The two alternative orders have total score larger than twice the original contribution, contradicting optimality. This reduces each transition to $\mathcal{O}(1)$.

For the full solution, add sentinels 0 and $M + 1$ to A , and define gaps

$$B_i = A_{i+1} - A_i.$$

The problem becomes: repeat $N - 1$ times; on step i , delete either the leftmost or rightmost remaining element p of B and pay $p(N - i)$, minimizing total cost.

Fix the boundary between elements deleted from the left and from the right. Let the two resulting sequences be x and y , both preserving their internal order. The problem is to merge x and y so as to minimize the sum, over every element, of

$$\text{element value} \times \text{number of deeper elements that came from the other sequence}.$$

If both x and y are nondecreasing, the sorted merge is optimal.

When $x_i \geq x_{i+1}$, there exists an optimum where these two elements stay adjacent; moreover replacing them by their average does not change the optimum. Generalizing this, repeatedly replace every non-increasing run by its average, just like the pool-adjacent-violators algorithm, until the sequence becomes nondecreasing. Maintain these averaged runs by run-length encoding.

A segment tree stores counts, sums, and cross-contributions for values from x and y . As the boundary moves, the run representation changes by only $\mathcal{O}(N)$ total insertions/deletions, and each is processed by the segment tree. Thus the full solution runs in $\mathcal{O}(N \log N)$ time.

Problem G. Many Common Segment Problems

For the first subtask with all $L_i, R_i \geq 1$, sort intervals so that

$$R_1 \leq R_2 \leq \dots \leq R_N.$$

If k is the minimum index among the chosen intervals, then their common intersection is nonempty iff every chosen interval has $L \leq R_k$. Maintain the distribution of L values with a BIT while iterating k in decreasing order. This gives $\mathcal{O}(N \log N)$.

For the general problem, temporarily allow choosing the empty set for each of the N interval groups; subtract the invalid original cases at the end. For a chosen set of k intervals, define its score as follows:

- add 1 for every integer x contained in all k intervals;
- subtract 1 for every adjacent pair $x, x + 1$ both contained in all k intervals.

This score is 1 if the chosen intervals have a common intersection and 0 otherwise.

For a fixed x , let P_i be the number of possible intervals for the i -th input item, and let Q_i be the number of such intervals containing x . Then the contribution of x is

$$\prod_{i=1}^N (P_i + Q_i).$$

The adjacent-pair term is similar. A direct implementation is $\mathcal{O}(NM)$.

To optimize, handle the easy cases $(L_i, R_i) = (-1, -1)$ and $L_i, R_i \neq -1$ separately. For a constraint $(-1, k)$, when $x \leq k$ the factor is $k + (k - x + 1)$, and when $x > k$ it is k . These are products of linear functions evaluated at many points. Use multipoint evaluation, while processing the extra condition $x \leq k$ inside a segment tree decomposition. By pushing the products of excluded leaves during the divide-and-conquer evaluation, the conditional part is handled without increasing the asymptotic complexity. Doing the same for $(k, -1)$ and for the $x, x + 1$ terms gives

$$\mathcal{O}(N \log^2 N + M \log^2 M).$$

Problem H. Music Game

Buttons whose success probability is 1 may be postponed to the end. Consider two remaining buttons: button A takes a seconds and succeeds with probability $b \neq 1$, and button B takes c seconds and succeeds with probability $d \neq 1$. Suppose x seconds have already elapsed.

If we press A before B , the expected time is

$$\frac{\frac{x+a}{b} + c}{d} = \frac{x + a + bc}{bd}.$$

If we press B before A , it is

$$\frac{\frac{x+c}{d} + a}{b} = \frac{x + c + ad}{bd}.$$

The first order is no worse iff

$$a(1-d) \leq c(1-b) \iff \frac{a}{1-b} \leq \frac{c}{1-d}.$$

Therefore sort all buttons by

$$\frac{T_i}{1 - A_i/B_i}$$

in nondecreasing order. The complexity is $\mathcal{O}(N \log N)$.

Problem I. Left Equals Right

Let

$$S = A_1 + A_2 + \cdots + A_N.$$

If S is odd, the two sides cannot be equal, so the answer is 0.

Assume S is even and put $X = S/2$. In every valid permutation, there is exactly one split point i such that

$$A_{P_1} + \cdots + A_{P_i} = A_{P_{i+1}} + \cdots + A_{P_N} = X.$$

Let C_i be the number of permutations whose valid split is after the first i elements. If T is a subset of indices of size i with sum X , then the first i positions may contain the elements of T in $i!$ orders and the remaining elements in $(N-i)!$ orders. Hence

$$C_i = \#\{T : |T| = i, \sum_{x \in T} A_x = X\} \cdot i!(N-i)!.$$

It remains to count, for every size, subsets with sum X . Use

$$\text{dp}[i][j][k] = \#\{T \subseteq \{1, \dots, i\} : |T| = k, \sum_{x \in T} A_x = j\}.$$

The complexity is $\Theta(N^2 X) = \Theta(N^3 A_{\max})$.

Problem J. Again Permutation Problem

For $N \leq 6$, there are at most $N! \leq 720$ possible permutations. Build the graph whose vertices are permutations, run BFS over reachable permutations, and sum their inversion counts. This costs $\mathcal{O}(N!(N^2 + M))$.

For the full solution, use the following group-closure notation. For a set of permutations P , let $F(P)$ be the set of permutations obtainable by composing elements of P any number of times. Let $F_i(P)$ be the subset of $F(P)$ whose first i positions are fixed as $(1, 2, \dots, i)$. We maintain a generating set P_i such that $F_i(P) = F(P_i)$.

The goal is to construct sets $Q_1, Q_2, \dots, Q_N$ such that $|Q_i| \leq N$, the product

$$Q_1 \times Q_2 \times \dots \times Q_N$$

has no duplicate elements, and equals $F(P)$. Once this factorization is available, a DP over the choices from $Q_1, \dots, Q_k$ can sum inversion contributions in $\mathcal{O}(N^4)$.

To move from P_k to P_{k+1} , enumerate the orbit of the value in position $k+1$ under $F(P_k)$ by BFS in $\mathcal{O}(N|P_k|)$ time. For every value x in this orbit, choose one representative permutation g_x whose $(k+1)$ -st position is x , and let G be the set of representatives. Define

$$P_{k+1} = \{g_{s(u)}^{-1} s g_u : u \in \text{orbit}_k, s \in P_k\},$$

where $s(u)$ denotes the image needed to keep the first $k+1$ positions fixed. Then $F(P_k) = G \times F(P_{k+1})$, so set $Q_{k+1} = G$.

The naive size of P_{k+1} may blow up. Compress any generating set S into a set T with $|T| \leq N^2$ and $F(S) = F(T)$ as follows. Prepare an $N \times N$ table d , initially empty. For each $s \in S$, scan $i = 1, 2, \dots, N$: if $d[i][s_i]$ is empty, store s there and stop; otherwise replace s by $s^{-1}d[i][s_i]$ and continue. The nonempty table entries form T . This process works in $\mathcal{O}(|S|N^2)$ time.

Combining these steps gives an $\mathcal{O}(N^6)$ solution.

Problem K. Peace with Magic

Suppose the final heights are $X_1, X_2, \dots, X_N$, and set $X_0 = X_{N+1} = 0$. Define

$$Y_i = X_i - X_{i-1} \quad (1 \leq i \leq N+1).$$

The number of spells needed to realize the heights can be reformulated as follows. Start with $y_1, \dots, y_{N+1} = 0$, and in one operation choose $i < j$ and add 1 to y_i while subtracting 1 from y_j . The original restriction about zeros between i and j can be ignored by performing operations from outside to inside.

In an optimal sequence, each y_i is only increased or only decreased. Otherwise, if operations (i, j) and (j, k) both occur for $i < j < k$, they can be replaced by a single operation (i, k) , contradicting minimality. Therefore the minimum number of spells is the total positive variation

$$\sum_{i=1}^{N+1} \max(X_i - X_{i-1}, 0).$$

Now use DP over the heights. Let

$$\text{dp}[i][j] = \text{minimum positive variation after deciding } X_1, \dots, X_i \text{ with } X_i = j.$$

Equivalently, write the transition as

$$\text{dp}[i][j] = \min \left(\min_{0 \leq k \leq j - D_{i-1}} \{\text{dp}[i-1][k] + (j - k)\}, \min_{j + D_{i-1} \leq k \leq j_{\max}} \{\text{dp}[i-1][k] + (k - j)\} \right),$$

with initialization $\text{dp}[0][0] = 0$ and $j_{\max} = D_1 + \dots + D_N \leq ND_{\max}$. Prefix and suffix minima over k compute every layer in linear time in the height range, so the total complexity is $\Theta(N^2 D_{\max})$.

Problem L. Construction of Town

Consider the multiset

$$D = \{d(i, j) : 1 \leq i < j \leq N\}.$$

A necessary condition is that the number of pairs at distance 1 is at most M . To minimize

$$\sum_{e \in D} A_e,$$

we would like to make exactly M distances equal to 1 and all remaining distances equal to 2.

This lower bound is attainable. Start with a star-shaped graph, and add $M - (N - 1)$ additional edges. Then exactly the desired number of vertex pairs have distance 1, and every other pair has distance 2. The construction and output can be done in $\mathcal{O}(N^2)$ time.

Problem M. Admired Person

Sort B in nondecreasing order:

$$B_1 \leq B_2 \leq \dots \leq B_M.$$

There is always an optimal sequence C that is also nondecreasing. If $i < j$ but $C_i > C_j$, then

$$|B_i - C_i| + |B_j - C_j| \geq |B_i - C_j| + |B_j - C_i|,$$

so swapping C_i and C_j does not increase the cost. Repeating this makes C sorted.

Now sort A as well, and match $B_1, B_2, \dots, B_M$ to a subsequence of A in order. Use

$$\text{dp}[i][j] = \text{minimum cost after considering } A_1, \dots, A_i \text{ and matching } B_1, \dots, B_j.$$

The initialization is $\text{dp}[0][0] = 0$, and the transition is

$$\text{dp}[i][j] = \min(\text{dp}[i-1][j], \text{dp}[i-1][j-1] + |A_i - B_j|).$$

The answer is $\text{dp}[N][M]$. The complexity is $\Theta(NM + N \log N + M \log M)$.

Problem N. Product Matrix

Evaluate the desired polynomial $f(x)$ at the geometric points

$$x = 1, 2, 4, \dots, 2^M.$$

Let $P_k = P(2^k)$. Then $f(2^k)$ is the $(1, 1)$ entry of

$$P_k P_{k+1} \dots P_{k+M-1}.$$

By precomputing cumulative products to the left and right of P_{M-1} and P_M , each value can be obtained in $\mathcal{O}(N^3)$ after $\mathcal{O}(N^3 M)$ preprocessing.

It remains to interpolate a degree- M polynomial from values at $2^0, 2^1, \dots, 2^M$. General multipoint interpolation is possible in $\mathcal{O}(M \log^2 M)$, but here the points form a geometric progression, and interpolation can be done in $\mathcal{O}(M \log M)$.

The Lagrange formula is

$$f(x) = \sum_{i=0}^M v_i \prod_{j \neq i} \frac{x - 2^j}{2^i - 2^j}, \quad v_i = f(2^i).$$

The denominators $\prod_{j \neq i} (2^i - 2^j)$ can all be enumerated in $\mathcal{O}(M)$. After setting

$$w_i = \frac{v_i}{\prod_{j \neq i} (2^i - 2^j)},$$

we need

$$\sum_{i=0}^M w_i \prod_{j \neq i} (x - 2^j) = \left(\prod_{i=0}^M (2^i - x) \right) \left(\sum_{i=0}^M \frac{w_i}{2^i - x} \right).$$

The product part is built by doubling in $\mathcal{O}(M \log M)$, or even $\mathcal{O}(M)$ with the Cauchy binomial theorem. For the rational-sum part, rewrite

$$\sum_{i=0}^M \frac{w_i}{2^i - x} = \sum_{j \geq 0} \left(\sum_{i=0}^M \frac{w_i}{2^{(i+1)j}} \right) x^j.$$

The inner sums are evaluations of a polynomial at 2^{-k} and can be listed by the chirp z -transform, i.e. by one convolution after separating the terms using $t_i = i(i-1)/2$.

Convolutions modulo $10^9 + 7$ can be performed through CRT with NTT-friendly moduli. The total complexity is

$$\mathcal{O}(N^3 M + M \log M).$$

Problem O. New School Term

Process pairs in the order $i = M, M-1, \dots, 1$. Greedily try to put students A_i and B_i into different classes whenever doing so is still possible.

Without the constraint that both classes have exactly N students, this is just a parity Union-Find. Add a distance-1 edge between A_i and B_i if they are not already connected; after all operations, each connected component is bipartitioned by parity of distance from an arbitrary root.

With the equal-size constraint, adding such an edge may make it impossible to choose exactly N students for one class. For every connected component c , let (x_c, y_c) be the two parity-class sizes. Feasibility is the subset DP asking whether one can choose either x_c or y_c from every component and sum to N .

The naive implementation runs this DP after every edge and costs $\Theta(N^2 M)$. To optimize, observe that if adding a distance-1 edge is impossible, adding a distance-0 edge is safe. For two components s, t , if merging them as $(x_s + y_t, y_s + x_t)$ is impossible, then the other merge $(x_s + x_t, y_s + y_t)$ is forced, and conversely. Thus the number of full feasibility recomputations is at most $N - 1$.

The DP itself can also be updated in $\Theta(N)$ per merge. Interpret

$$\text{dp}[j] = [x^j] \prod_c (x^{x_c} + x^{y_c}).$$

When two components are merged, divide by their two old factors and multiply by the new factor. The division is easy when $x_c = y_c$, and otherwise uses the series

$$\frac{1}{1+x} = 1 - x + x^2 - x^3 + \dots$$

This is the standard “restoring DP” technique. Values are stored modulo a randomly chosen large prime, for example from $[10^{17}, 10^{18}]$, to avoid enormous integers; the failure probability is negligible.

The final complexity is

$$\Theta(N^2 + M\alpha(N)).$$

Problem P. Make Testcase

Let $B = (B_1, B_2, \dots, B_N)$ be the frequency distribution of the sequence A . The key fact is that the answer to Shuffle and Increasing is unchanged if we permute the entries of B .

To see this, solve Shuffle and Increasing in terms of auxiliary values f_k . For fixed positions $i_1 < i_2 < \dots < i_k$, the corresponding count is equivalent to partitioning the multiset A into $N - k$ specified strictly increasing sequences. This depends only on the multiset of frequencies, not on the order in which the frequencies appear. Since the original answers g_i are related to the f_k values by

$$f_k = \sum_{i=k}^{N-1} \binom{i}{k} g_i,$$

knowing all f_k determines all g_i . Therefore permuting B does not change the answer.

Thus we may assume B is nondecreasing. For a fixed N , there are only $p(N)$ such frequency distributions, where $p(N)$ is the partition number. For each candidate distribution, solve the original problem directly, for example by insertion DP, in $\mathcal{O}(N^2)$ time. Enumerating all possible test cases and collecting all achievable answers costs

$$\mathcal{O}(N^3 p(N)).$$

Then each query X can be answered by outputting a precomputed test case, with total additional cost $\mathcal{O}(TN)$.